

CPP NOTES – DAY 33

Upcasting

Upcasting is something that occurs automatically, without us having to manually cast it. It occurs when we create a Base Class pointer, and treat it like a pointer to the Child Class.

In short, Upcasting occurs when we attempt to cast a Child to a Parent, “Up” the Hierarchy. Downcasting is the exact opposite.

This allows us to be able to access the functions within the Child class, which the Base Class Pointer should not be able to access normally. Of course, we must have the virtual keyword enabled on the function(s) of the Base class, otherwise the effect will be different.

```
1  #include <iostream>
2  using namespace std;
3
4
5  class Employee {
6  public:
7      string name;
8
9      virtual void display() {
10         cout << "I am an Employee" << endl;
11     }
12 };
13
14 class Manager : public Employee{
15     string branch;
16
17 public:
18     void display() {
19         cout << "I am an Manager" << endl;
20     }
21 };
22
23 class Receptionist : public Employee {
24 public:
25     int lobbyNumber;
26
27     void Lobby() {
28         cout << "Lobby: " << lobbyNumber << endl;
29     }
30 };
31
32 int main() {
33     Manager m1;
34     m1.name = "Mark";
35
36     Employee* e1 = &m1; // Upcasting
37
38     e1->display();        // Base Class Pointer
39 }
```

Virtual Table (vtable)

is a mechanism used to support dynamic (runtime) dispatch of virtual functions.

- It is essentially a lookup table used by objects of a class to determine which function to invoke at runtime when a virtual function is called.
- It allows programs to decide at runtime which version of a function to execute, especially when dealing with inheritance and polymorphism.

Why Virtual Table:

Consider you have a base class pointer pointing to a derived class object. If you call a virtual function using that pointer, how will the program know to call the derived class's overridden function instead of the base class's? **This is where the vtable comes in.**

Working -> Base has a vtable with one entry: pointer to Base::show.

1. Derived has its own vtable with the same entry overridden: pointer to Derived::show.
2. When `b->show()` is called:
 - `b` has a hidden `vp`tr pointing to Derived's vtable.
 - The correct function (Derived::show) is resolved at runtime using the vtable.

Virtual Pointer

The virtual pointer (`vp`tr) is a hidden pointer inside an object of a class that has virtual functions. It points to the virtual table (vtable) of the object's actual class type. This pointer is automatically managed by the compiler and is crucial for runtime polymorphism.

Why vptr?

When you call a virtual function through a base class pointer/reference that points to a derived class object, the compiler can't determine at compile time which function to call. The vptr helps in resolving this at runtime.

It does this by:

1. Pointing to the correct vtable for the object.
2. Allowing dynamic dispatch via the vtable entries.

Templates

C++ template is a powerful tool that allows you to write a generic code that can work with any data type. The idea is to simply pass the data type as a parameter so that we don't need to write the same code for different data types.

For example, same sorting algorithm can work for different type, so rather than writing and maintaining multiple codes, we can write one sort() and pass the datatype as a parameter.

Eg; template <class T1,class T2>

File I/O (Input/Output) in C++ allows programs to read from and write to files using the <fstream> library, which provides three main classes:

1. ifstream – for reading from files (input).
2. ofstream – for writing to files (output).
3. fstream – for both reading and writing.

◆ Basic File I/O Example

Writing to a File

```
#include <iostream>

#include <fstream>

using namespace std;

int main() {

    ofstream outFile("example.txt"); // Create and open a file for writing

    if (outFile.is_open()) {

        outFile << "Hello, file I/O in C++!" << endl;

        outFile << "This is a second line." << endl;

        outFile.close(); // Close the file

    } else {

        cout << "Unable to open file for writing." << endl;

    }

    return 0;

}
```

Reading from a File

```
#include <iostream>

#include <fstream>

#include <string>

using namespace std;


int main() {

    ifstream inFile("example.txt"); // Open file for reading

    string line;


    if (inFile.is_open()) {

        while (getline(inFile, line)) {

            cout << line << endl; // Output each line to the console

        }

        inFile.close(); // Close the file

    } else {

        cout << "Unable to open file for reading." << endl;

    }


    return 0;

}
```

◆ Using fstream for Both Read and Write

```
#include <iostream>

#include <fstream>

using namespace std;

int main() {

    fstream file("example.txt", ios::in | ios::out | ios::app); // Open for
    read/write and append

    if (file.is_open()) {

        file << "Appending a new line.\n";

        file.seekg(0); // Move read pointer to the beginning

        string line;
        while (getline(file, line)) {

            cout << line << endl;

        }

        file.close();

    } else {

        cout << "Unable to open file." << endl;

    }

    return 0;
```

}

◆ File Opening Modes

Mode	Description
ios::in	Open for reading
ios::out	Open for writing
ios::app	Append to the end of the file
ios::trunc	Truncate file if it exists
ios::binary	Binary mode

FYI:

- **Dynamic dispatch** is a mechanism in cpp where the correct version of virtual function is called at runtime based on the actual type of the object. Also known as late binding.
- An **abstract class** in C++ is a class that can be used only as a base class of some other class. It contains at least one pure virtual function, which is a function that has no definition.
- A **pure virtual function** is a special type of virtual function in C++ that has no implementation in the base class and must be overridden by derived classes. It's used to define abstract interfaces in object-oriented programming (functions which is mandatory to be adapted by the derived class).
- **Virtual destructor** - When a base class pointer points to a derived class object and you delete it, the derived class's destructor must be called first — otherwise, it may cause:
 - Resource leaks (memory not freed)
 - Incomplete destruction (sub object state not cleaned up)
 - The process of raising an exception is called throwing.
 - `catch(...)` – default catch (atlast)